ENCONTRO 6 DE 6

Do simulador ao silício

Curso Introdutório de Programação de Baixo Nível



```
sw  $t1, 8($s7)
```

soma = 41

Liga ProtoFPGA · UFSC Araranguá

DE ONDE PARAMOS

As três perguntas da semana passada

1. **Lá não há sistema operacional. Quem atende o syscall?**

   Ninguém. O programa fala direto com o hardware, com lw e sw.

2. **Sem Run I/O, como saber se o programa funcionou?**

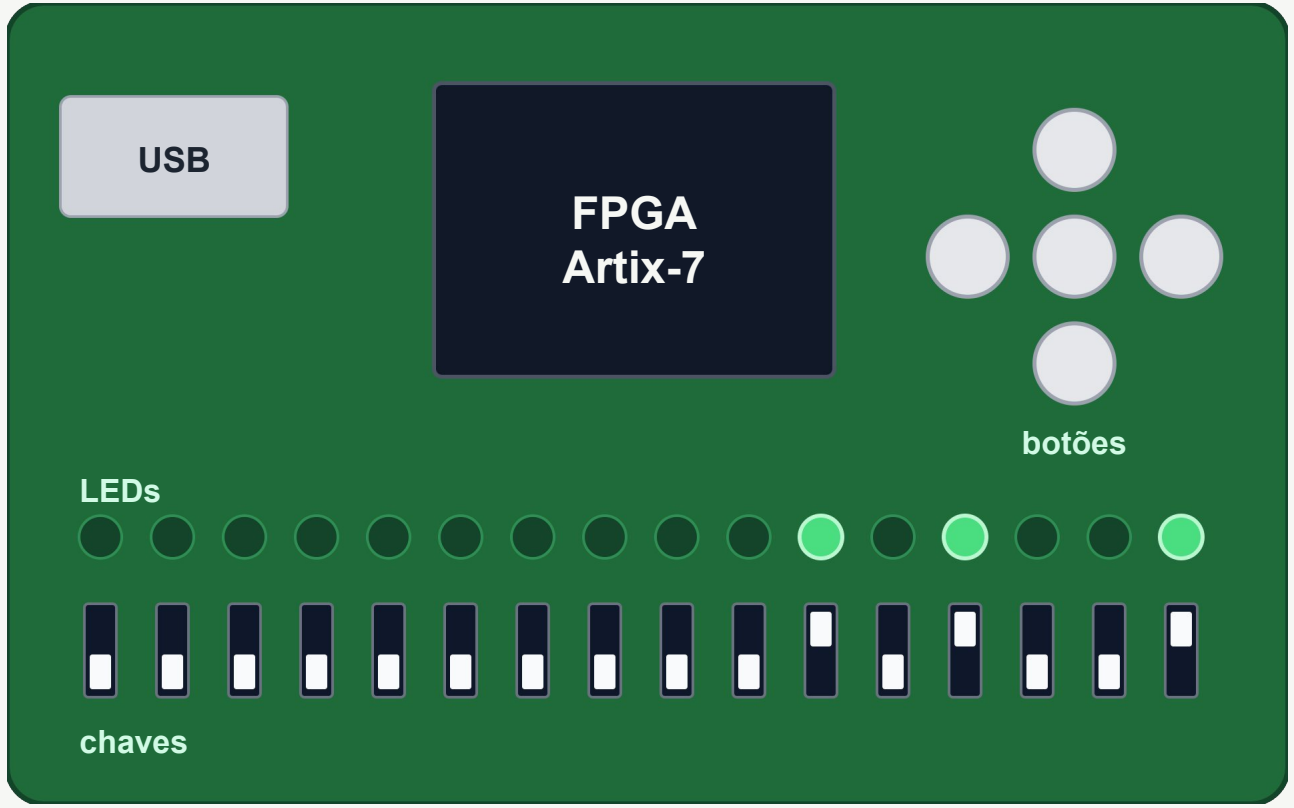
   Pelos 16 LEDs da placa.

3. **Todas as instruções que vocês usaram existem lá?**

   Não. O processador da placa tem só 37.

A PLACA

Um FPGA e o que tem em volta dele



FPGA: um chip cheio de pequenas tabelas-verdade e flip-flops, ligados por fios programáveis. Ele vira o circuito que você descrever.

16 LEDs — a nossa “tela”

16 chaves — um número

5 botões — comandos

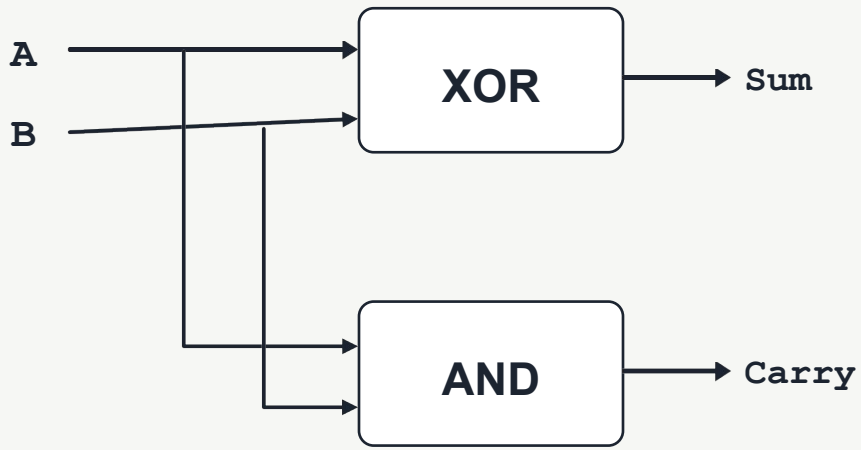
reset — recomeça

HDL

VHDL descreve um circuito

```
entity halfadd is
  port(A, B         : in  std_logic;
       Sum, Carry : out std_logic);
end halfadd;

architecture comp of halfadd is
begin
  Sum   <= A xor B;
  Carry <= A and B;
end comp;
```



Não há “linha seguinte”: as duas linhas são duas portas que existem ao mesmo tempo, o tempo todo.

O PROCESSADOR

MIPS_S: um MIPS de verdade, em VHDL

32 bits	os mesmos 32 registradores do MARS
37 instruções	um subconjunto do MIPS (o S é de subset)
multiciclo	quase toda instrução leva 4 ciclos de clock
mesmo código	o código de máquina do MARS roda igual lá

Registradores, lw e sw, laços, funções, pilha: tudo vale lá. Só muda o que o processador sabe fazer.

O SUBCONJUNTO

O que ficou de fora, e por quê

`add addi sub`	Fazem a mesma conta que addu, addiu e subu. Só geram uma exceção no transbordo — e o MIPS_S não trata exceções.
`mul div`	Multiplicar de uma vez gasta muito hardware. O MIPS_S tem multu e divu, que fazem um bit por ciclo.
`syscall`	Pede ajuda ao sistema operacional. Na placa não há nenhum.

Cada instrução a mais é hardware a mais. O MIPS_S fica só com o necessário — e assim é pequeno e fácil de entender.

O QUE ESCREVER NO LUGAR

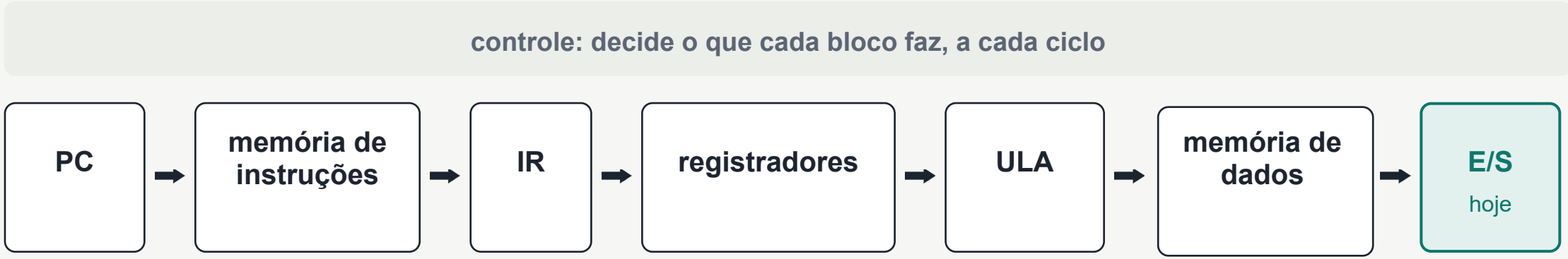
Trocas para rodar no MIPS_S

Em vez de	Escreva
`add $t1, $t1, $t4`	`addu $t1, $t1, $t4`
`addi $t0, $t0, 1`	`addiu $t0, $t0, 1`
`sub $t2, $t0, $t1`	`subu $t2, $t0, $t1`
`mul $v0, $a0, $v0`	`multu $a0, $v0` e `mflo $v0`
`syscall`	`lw` / `sw` nos endereços de E/S

Na dúvida, olhe a coluna Basic do MARS: é ela que vai para a placa.

POR DENTRO, POR CIMA

O diagrama de blocos, simplificado



Desvios: o PC recebe um endereço novo. lw: o dado da memória volta para um registrador.

Cada bloco vocês já viram no MARS. O MARS simula exatamente isto.

SEM SYSCALL

Entrada e saída são lw e sw

0xFFFF0000	botões	lw
0xFFFF0004	chaves	lw
0xFFFF0008	LEDs	sw

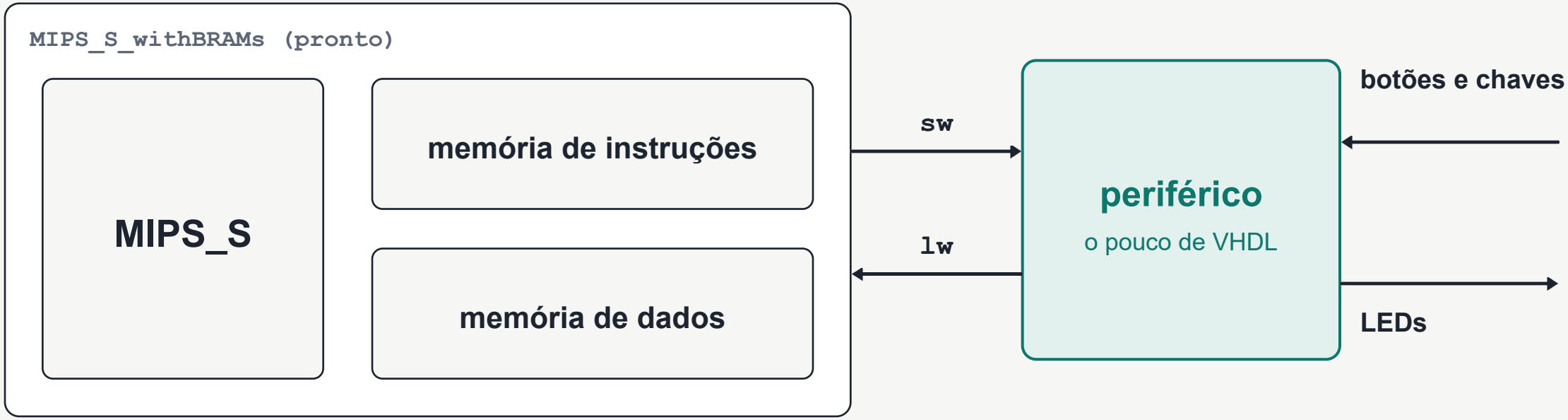
```
lui  $s7, 0xFFFF  # base
lw   $t0, 0($s7)  # botões
lw   $v0, 4($s7)  # chaves
sw   $a0, 8($s7)  # LEDs
```

Botões: bit 0 = C, 1 = U, 2 = L, 3 = R, 4 = D. Para testar um, andi com a máscara (C = 0x01 … D = 0x10).

Para o processador, é só memória. Quem decide que 0xFFFF0008 acende LEDs é o circuito ligado a ele: o periférico.

O SISTEMA NA PLACA

Processador, memórias e periférico



O periférico é um multiplexador (para o lw) e um registrador (para o sw). O programa do MARS vai para as memórias.

O PROGRAMA DE HOJE

aula6_contador.asm

```
laco:  jal   espera_botao  # $v0 = botão
       andi  $t0, $v0, 0x02  # BTNU?
       beq   $t0, $zero, nao_u
       addiu $s0, $s0, 1
nao_u: andi  $t0, $v0, 0x10  # BTND?
       beq   $t0, $zero, nao_d
       addiu $s0, $s0, -1
nao_d: … BTNL, BTNR, BTNC
nao_c: addu  $a0, $s0, $zero
       jal   mostra_leds
       j     laco  # nunca termina
```

BTNU	+ 1
BTND	− 1
BTNL	× 2
BTNR	÷ 2
BTNC	= chaves

Sem sistema operacional, não há para onde voltar: o programa nunca termina.

DO BOTÃO AO LED

Um aperto de BTNU, passo a passo



O seu código nunca toca no botão nem no LED: ele só faz lw e sw. O periférico é o circuito no meio do caminho, que liga esses endereços aos pinos da placa. No MARS, quem fazia esse papel era o syscall.

DO MARS À PLACA

Primeiro no MARS, depois na placa

1. **Testar no MARS**
   aula6_contador_mars.asm: as funções de E/S usam o teclado e o Run I/O

2. **Trocar as funções de E/S**
   mostra_leds vira um sw; le_chaves, um lw — o resto do programa não muda

3. **Gerar a placa**
   Dump Memory → memory.vhd → Vivado → bitstream (alguns minutos)

É a convenção da aula 5 trabalhando: mesmos nomes, mesmos $a e $v0. O laço não sabe se está na placa ou no simulador.

VAMOS FAZER JUNTOS

O programa do encontro 4 roda no MIPS_S?

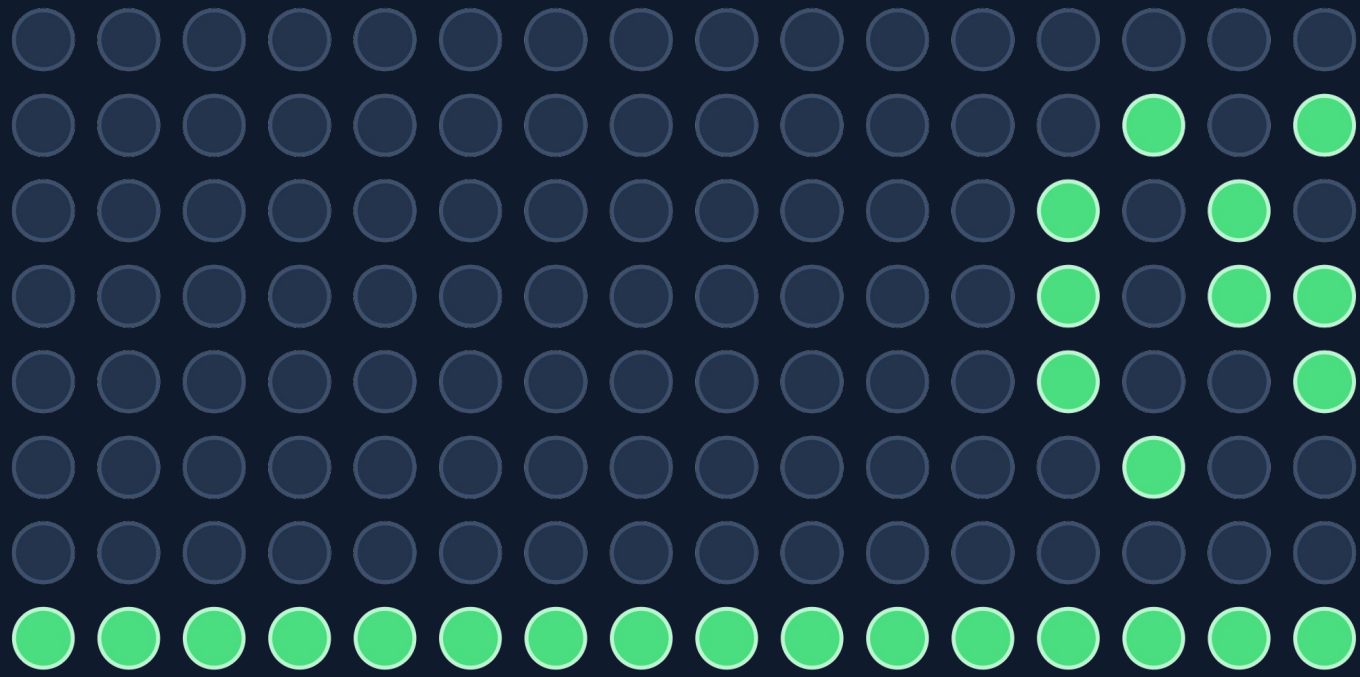
```
laco: slt  $t3, $t0, $s1
      beq  $t3, $zero, fim
      lw   $t4, 0($s0)
      add  $t1, $t1, $t4
      …
pula: addi $t0, $t0, 1
      addi $s0, $s0, 4
      j    laco
fim:  li   $v0, 4
      la   $a0, msg1
      syscall  # e mais 4
```

1. Quais linhas não existem no MIPS_S?
2. O que escrever no lugar?
3. Sem syscall, como mostrar a soma?
4. E como terminar?

NA PLACA

Demonstração

reset	0
SW = 5, BTNC	5
BTNL	10
BTNU	11
BTND ×2	9
BTNR	4
SW = 0, BTNC	0
BTND	−1



PARA FECHAR

O mesmo código, do simulador ao silício

- O MIPS_S tem 37 instruções: add vira addu, mul vira multu e mflo.
- Sem syscall, entrada e saída são lw e sw em endereços especiais.
- Na placa, o programa nunca termina.

Projeto final: link do repositório no GitHub até 19/11, às 23h59.

Obrigado! Liga ProtoFPGA · UFSC Araranguá